Parcours des écoles d'ingénieurs Polytech (MAM3)

Introduction à la Programmation et à l'Algorithmique



Dino Lopez Pacheco, Hélène Renard-Ferry et al

À quoi sert la programmation ?



- ✓ Passer d'un problème réel à une résolution informatique :
  - ❑ Calculs mathématiques
  - ❑ Gestion de commandes
  - ❑ Systèmes d'exploitation
  - ❑ *Facebook*, *twitter*
  - ❑ Jeux
- ✓ Analyser le problème, concevoir une solution, l'écrire dans un langage informatique, l'exécuter sur une machine.

Environnements de programmation



✓ Écrire un programme, le transformer en code binaire, l'exécuter, le documenter, le tester …



L'outil ***Doxygen*** *extrait l'information à partir du code source et bien plus encore …*

✓En Java :

- ❑ Éditeurs classiques : *emacs*, *kwrite* … et + commandes systèmes
- ❑ Environnements de programmation : *Éclipse*, *IntelliJ* …

Objectifs du cours



- ✓ Apprendre les principes de base de la programmation objet et de l'algorithmique;
- ✓ Être capable de comprendre un code;
- ✓ Être capable d'écrire un code « propre » : lisible, maintenable;
- ✓ Savoir écrire un petit logiciel en Java.

Contenu du cours



- ✓ Introduction aux objets;
- ✓ Algorithmes de base;
- ✓ Structures de données;
- ✓ Listes;
- ✓ Tris de données;
- ✓ Qualité du code.



Qu'est-ce qu'un algorithme ?

Un algorithme est une méthode générale pour résoudre un ensemble de problèmes.

Problèmes, Algorithmes & Programmes



- ✓ Un **problème** est une tâche qui doit être effectué
  - ❑ plusieurs contraintes
  - ❑ état initial -> sortie recherché.
- ✓ Un **algorithme** est une méthode générale pour résoudre un ensemble de problèmes.
  - ❑ Un algorithme doit être correcte, composé d'un nombre fini des étapes précises vers la solution du problème, sans ambiguïté, et son exécution doit finir dans un instant donné. Un algorithme est dit correct lorsque, pour chaque instance du problème, il se termine en produisant la bonne sortie
  - ❑ On mesure l'efficacité d'un algorithme notamment par sa durée de calcul, par la précision des résultats etc.
  - ❑ L'analyse de la complexité algorithmique permet de prédire l'évolution en temps calcul nécessaire pour amener un algorithme à son terme, en fonction de la quantité de données à traiter.
- ✓ Un **programme** est la réalisation concrète d'un algorithme.

Définition du problème

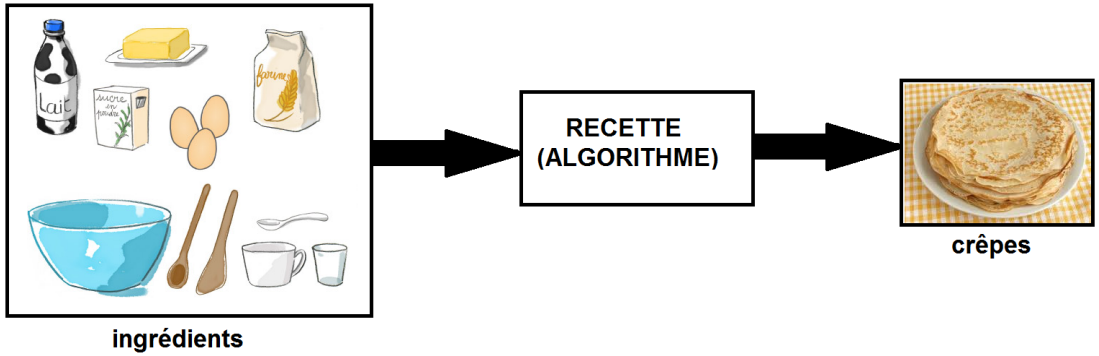
✓ Faire des crêpes

❑ Définition des objets manipulés : « Comment faire des crêpes sucrées ? »



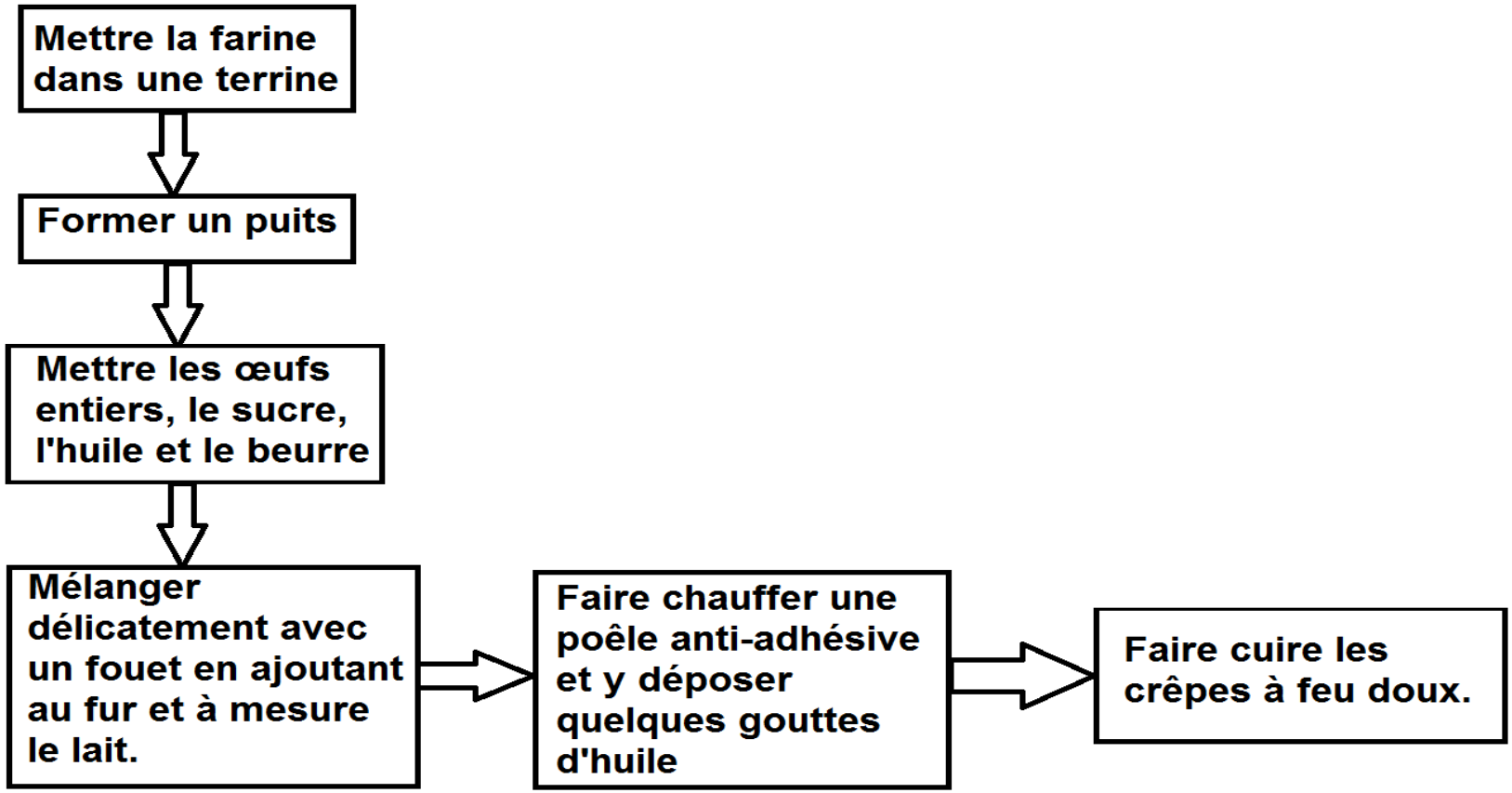
Chaque mot a son importance.

❑



Des ingrédients aux crêpes, images honteusement chipées sur le blog de hoptoys

❑ **<u>Ordonner</u>** la liste des opérations :



Vers la construction d'un algorithme (1/2)

- ✓ Définition du problème
  - ❑ comment maintenir un four à la température souhaitée par un utilisateur ?
- ✓ Enoncé de la solution
  - ❑ Voir quelle température a été choisi par l'utilisateur. Ensuite, si la température du four est bas par rapport à la température souhaitée, activer fortement la résistance chauffante du four. Si la température est égale à la température souhaitée, activer très faiblement la résistance chauffante du four afin d'empêcher son refroidissement. Enfin, si la température du four est supérieure à la température souhaitée, désactiver la résistance chauffante. Après avoir pris une décision sur l'activation ou désactivation du four, atteindre 60 seconds, puis répéter la même opération tant que le four est allumé.

Vers la construction d'un algorithme (1/2)

- ✓ identifier les variables du système.
  - ❑ Température demandée par l'utilisateur du four : Tu
  - ❑ Température du four : Tf
  - ❑ Intervenir un laps de temps plus tard : t
- ✓ Généralisation de l'énoncé de la solution avec des variables
  - ❑ Lire la température Tu choisi par l'utilisateur. Ensuite, si la température du four Tf est bas par rapport à Tu, activer fortement la résistance chauffante du four. Si Tf est égal à Tu, activer très faiblement la résistance chauffante du four afin d'empêcher son refroidissement. Enfin, si Tf est plus grand que Tu, désactiver la résistance chauffante. Après avoir pris une décision sur l'activation ou désactivation du four, atteindre t secondes (60s), puis répéter la même opération tant que le four est allumé.

L'algorithme en pseudocode



- Simplification de phrases
- Langage proche de celle de la machine
  - Utilisation de fonctions propres à la machine
  - Utilisation de fonctions à définir (sous-problème)
- Pseudocode

```
Fonction lancement_four(){
  Tu : température du four souhaité par l'utilisateur
(entier)
  Tf : température courante du four (entier)
  t = 60 : temps en sec avant de vérifier l'état
actuel du four (entier)
  Lire Tu
  Tant que (four_est_allumé()) {
    Lire Tf
    Si (Tf < Tu) {
      activer_resistance_fortement()
    } Sinon {
      Si (Tf == Tu) {
        activer_resistance_faiblement()
      } Sinon {
        desactiver_resistance()
      }
    }
    Attendre(t)
  }
```

Algorithme : problèmes



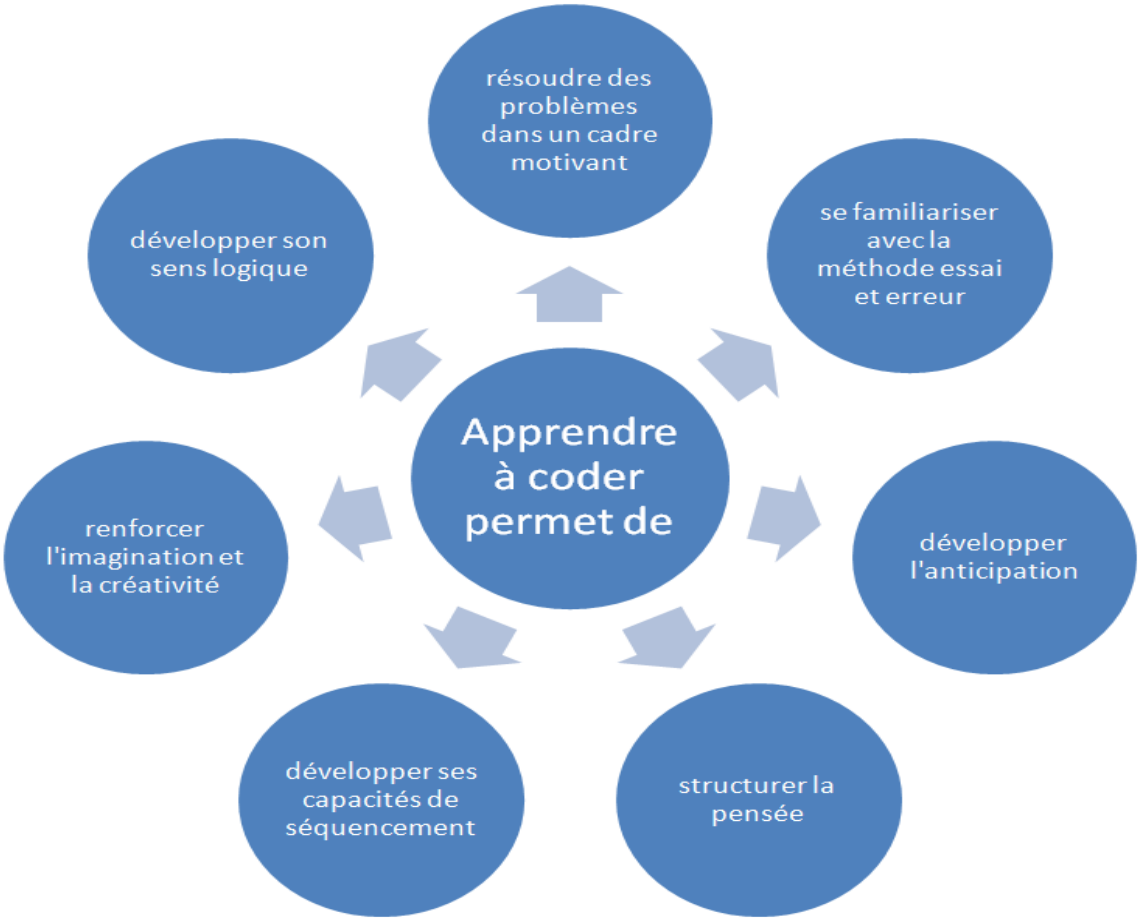
✓ <u>La correction</u> :

- ❑ Mon algorithme termine-t-il ?
- ❑ Mon algorithme est-il correct ?

✓ <u>La complexité</u> :

- ❑ Combien de temps va prendre l'exécution ?
- ❑ Mon algorithme est-il optimal ?

Pourquoi apprendre à coder ?



https://dane.ac-besancon.fr/le-codage-informatique-dans-les-nouveaux-programmes/

Écriture des pseudocodes

L'importance de l'algorithme



- ✓ Pour écrire un algorithme, 2 solutions sont souvent employés
  - ❑ Flowchart – diagramme en boîte et flèches
  - ❑ Pseudocode – langage humain, mais notation très proche d'un langage de programmation à haut niveau
- ✓ écrire l'algorithme est une étape très importante qui permet de gagner en efficacité (et donc, en temps de développement)
  - ❑ il n'y a pas d'erreurs de compilation
  - ❑ permet d'appréhender les contraintes du système à créer sans les contraintes du langage de programmation
  - ❑ évaluer la complexité de la solution
  - ❑ permet la parallélisassions de tâches
  - ❑ la continuité d'un projet ne dépend pas de la présence d'un développeur

Comment écrire du pseudocode



- Pas de règles bien précises
  - Mais ça doit être harmonisé autant que possible
- Quelques standards de facto ont été adopté
  - Enrichis selon les besoin du projet
- Quelques exemples suivent

```
fonction demande_age(nom) {
   affiche « Bienvenu. Aujourd'hui c'est le » + date()
   affiche « quel est votre âge M. » + nom + « ? »
   lire age ; // « age » est une variable contenant âge rentrée par l'utilisateur
}
```

```
fonction somme(a,b) {
   // si malgré tout, je pense que mon pseudocode n'est pas assez claire,
   // je peux introduire des commentaires à mon algorithme, comme ici
   // la variable c garde la somme
    c = a + b ; // commentaire dans la même ligne d'une instruction!!
   // et on renvoie le résultat
   retour c
}
```

Bloques conditionnels et *switch*




```
si (condition(s)) {
    …
} sinon { ; // sinon est
optionnel
    …
}

si (condition(s)) {
    …
} sinon si (condition(s)) {
    …
} ; // sinon ou rien, au besoin
```

```
Si (var) égal à
    « a ») { … faire quelque
chose}
    « b ») { … faire autre
chose}
    Autre chose {… comportement
par défaut}
Fin si
```

Les boucles




```
Faire tant que (condition(s)) {
    … ; // chose utile pour ma fonction
}

Faire {
} tant que (condition(s))

Pour (variable(s) initiale(s)), tant que
(condition(s)), avec (incrément(s)) {
 }
```

Tableaux

- Un tableau est une collection de données
  - Le premier élément est souvent supposé se trouver à l'index zéro
  - Et le dernier à l'index « n-1 »
- Tableau[i]
  - i-ième élément du tableau « Tableau »

- `tab = {} ;// tableau vide`
- `tab = {"a","b","c"} ; // tab[0] sera égal à « a »`
- `tab = {{1,"a"},{2,"b"},{3,"c"}} ; // tab[2,0] sera égal à « 3 »`
- `tab1 = tab[2] ;// équivalent à tab1 = {3,"c"}`
- `tab[2,0]=4 ; // on a changé « 3 » par « 4 »`



Complexité algorithmique

Performance d'un algorithme



- ✓ Plusieurs moyens d'évaluer la performance d'un algorithme
  - ❑ Temps de calcul
  - ❑ mémoire utilisée
  - ❑ précision de la sortie
- ✓ Le temps d'exécution d'un programme
  - ❑ intuitivement, cela dépend du matériel
  - ❑ certaines programmes peuvent être qualifiés comme lents depuis sa conception
- ✓ La complexité d'un algorithme ou programme
  - ❑ correspond au temps d'exécution (pas à la difficulté pour programmer le code)
  - ❑ notation « big-O » : O(f(x))

Temps d'exécution … en chanson




```
1. Fonction chante_allouette(partie[0..n-1]){
2.   Pour (i=0), tant que (i < n), avec (i++) {
3.     Chante « Alouette, gentille alouette »
4.     Chante « Alouette, je te plumerai »
5.     Pour (j=0), tant que (j < 2), avec (j++) {
6.       Chante « Je te plumerai » + partie[i]
7.     }
8.     Pour (j=i), tant que (j >= 0), avec (j--)
{
9.       Chante « Et » + partie[j] + « ! Et » +
partie[j] + « ! »
10.    }
11.    Chante « Alouette, Alouette! »
12.    Chante « Oh, oh, oh, oh »
13.  }
14.}
```

- Quelle temps d'exécution ?
  - Intuitivement, on compterait chaque instruction
    - Chaque « chante » = 1 unité
    - Accès en lecture = 1 unité
    - Accès en écriture = 1 unite
  - Cependant, en fait, ça dépend du nombre d'éléments du tableau « partie »
- Pour estimer la complexité d'un algorithme ou programme
  - Ignorer les constantes
    - Instructions « chante », lecture, écriture, etc.
  - Se focaliser sur les parties variables
    - Parties dépendantes de « n »

Temps d'exécution … en chanson

```
1. Fonction chante_allouette(partie[0..n-1]){
2.   Pour (i=0), tant que (i < n), avec (i++) {
3.     Chante « Alouette, gentille alouette »
4.     Chante « Alouette, je te plumerai »
5.     Pour (j=0), tant que (j < 2), avec (j++) {
6.       Chante « Je te plumerai » + partie[i]
7.     }
8.     Pour (j=i), tant que (j >= 0), avec (j--)
{
9.       Chante « Et » + partie[j] + « ! Et » +
partie[j] + « ! »
10.    }
11.    Chante « Alouette, Alouette! »
12.    Chante « Oh, oh, oh, oh »
13.  }
14.}
```

- 2 parties variables
  - Ligne 2 (boucle pour dès i = 0 jusqu'à n-1
  - Ligne 8 (dépendant de i)
- La ligne 5 est constante
- Estimation du nombre d'exécutions des bloques
  - $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$
- Lorsque n -> infini
  - La division par une constante n'a qu'un impacte limité
- Complexité : $O(n^2)$

Complexité algorithmique

- Réduire le temps d'exécution à l'un de ces termes
  - O(1) – constante
  - O(log n) – logarithmique
  - O(n) – linéaire
  - O(n log n) – n log n
  - $O(n^2)$ – quadratique
  - $O(n^k)$ – polynomiale (k >= 1)
  - $O(a^n)$ – exponentielle
- Règle générale : Une solution viable ne doit pas être exponentielle

